Definición de funciones en Python

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Tabla de contenidos

Introduction **3**

1 Flujo de ejecución, argumentos y parámetros **3**
1.1 Concepto de función . . . 3
1.2 Flujo de ejecución . . . 3
1.3 Argumentos vs Parámetros . . . 3

2 Definición de funciones **4**
2.1 Sintaxis básica . . . 4
2.2 Ejemplos básicos . . . 4
2.3 Función sin return . . . 5

3 Tipos de argumentos **6**
3.1 Argumentos posicionales . . . 6
3.2 Argumentos con nombre (keyword arguments) . . . 6
3.3 Argumentos con valores por defecto . . . 6
3.4 Argumentos variables (*args) . . . 7
3.5 Argumentos variables con nombre (**kwargs) . . . 7

4 Documentación de funciones **8**
4.1 Docstrings . . . 8
4.2 Anotaciones de tipo (Type hints) . . . 10

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

5 Scope de variables **10**
5.1 Variables locales vs globales . . . 10
5.2 Buenas prácticas . . . 11

6 Funciones lambda **12**
6.1 Sintaxis básica . . . 12
6.2 Aplicaciones económicas . . . 12
6.3 Lambdas en estructuras de datos . . . 13

7 Funciones map y filter **13**
7.1 Función map() . . . 13
7.2 Función filter() . . . 14
7.3 Combinación de map y filter . . . 15

8 Módulo NumPy **16**
8.1 Instalación e importación . . . 16
8.2 Creación de arrays . . . 16
8.3 Operaciones con arrays . . . 17
8.4 Estadísticas con NumPy . . . 18
8.5 Operaciones con matrices . . . 19
8.6 Generación de números aleatorios . . . 20

9 Módulo Pandas **21**
9.1 Instalación e importación . . . 21
9.2 Series de Pandas . . . 21
9.3 DataFrames . . . 23
9.4 Operaciones con DataFrames . . . 24
9.5 Lectura y escritura de datos . . . 25
9.6 Aplicaciones económicas con Pandas . . . 26

10 Aplicaciones económicas **27**
10.1 Aplicación 1: Calculadora financiera completa . . . 27
10.2 Aplicación 2: Análisis de portafolio con NumPy y Pandas . . . 29

11 Ejercicios prácticos **31**
11.1 Ejercicio 1: Sistema de predicción económica . . . 31
11.2 Ejercicio 2: Optimizador de producción . . . 33

12 Conlusión **34**

13 Próximos pasos **35**

14 Próximos pasos recomendados **35**

15 Recursos adicionales **35**

16 Publicaciones Similares **35**

Definición de funciones en Python

En esta sexta guía exploraremos las funciones en Python, una herramienta fundamental para escribir código modular, reutilizable y organizado. Las funciones son especialmente importantes en análisis económico, donde necesitamos realizar cálculos complejos repetidamente con diferentes datos.

1 Flujo de ejecución, argumentos y parámetros

1.1 Concepto de función

Una función es un bloque de código reutilizable que realiza una tarea específica. Las funciones ayudan a:

- Organizar el código en módulos lógicos
- Evitar repetición de código
- Facilitar el mantenimiento
- Hacer el código más legible
- Permitir la reutilización

1.2 Flujo de ejecución

```
# El flujo de ejecución con funciones

# 1. Definición de la función
def calcular_interes(capital, tasa):
    """Calcula el interés simple"""
    interes = capital * tasa
    return interes

# 2. Código principal se ejecuta línea por línea
print("Inicio del programa")

# 3. Al llamar la función, el flujo salta a ella
resultado = calcular_interes(10000, 0.05)

# 4. Después de ejecutar la función, el flujo regresa
print(f"Interés: S/ {resultado:,.2f}")

print("Fin del programa")
```

```
Inicio del programa
Interés: S/ 500.00
Fin del programa
```

1.3 Argumentos vs Parámetros

```
# Parámetros: variables en la definición de la función
def calcular_pib_per_capita(pib, poblacion):  # pib y poblacion son parámetros
    """Calcula PIB per cápita"""
    pib_pc = (pib / poblacion) * 1000000
    return pib_pc

# Argumentos: valores pasados al llamar la función
resultado = calcular_pib_per_capita(242632, 33715471)  # 242632 y 33715471 son argume
print(f"PIB per cápita: USD {resultado:,.2f}")
```

```
PIB per cápita: USD 7,196.46
```

2 Definición de funciones

2.1 Sintaxis básica

```
def nombre_funcion(parametros):
    """
    Docstring: Descripción de la función
    """
    # Cuerpo de la función
    # Código que realiza la tarea

    return resultado  # Opcional
```

2.2 Ejemplos básicos

```
# Función sin parámetros
def saludar():
    """Imprime un saludo"""
    print("Bienvenido al análisis económico")

saludar()

# Función con un parámetro
def calcular_igv(precio):
    """Calcula el IGV (18%) de un precio"""
    igv = precio * 0.18
    return igv

igv = calcular_igv(100)
print(f"IGV: S/ {igv:.2f}")

# Función con múltiples parámetros
def calcular_utilidad(ingresos, costos):
    """Calcula la utilidad"""
    utilidad = ingresos - costos
```

```
    return utilidad

utilidad = calcular_utilidad(100000, 75000)
print(f"Utilidad: S/ {utilidad:,.2f}")

# Función con múltiples valores de retorno
def analizar_ventas(ventas):
    """Retorna estadísticas de ventas"""
    total = sum(ventas)
    promedio = total / len(ventas)
    maximo = max(ventas)
    minimo = min(ventas)

    return total, promedio, maximo, minimo

ventas_mes = [45000, 52000, 48000, 55000, 60000]
total, prom, max_v, min_v = analizar_ventas(ventas_mes)

print(f"Total: S/ {total:,.2f}")
print(f"Promedio: S/ {prom:,.2f}")
print(f"Máximo: S/ {max_v:,.2f}")
print(f"Mínimo: S/ {min_v:,.2f}")
```

```
Bienvenido al análisis económico
IGV: S/ 18.00
Utilidad: S/ 25,000.00
Total: S/ 260,000.00
Promedio: S/ 52,000.00
Máximo: S/ 60,000.00
Mínimo: S/ 45,000.00
```

2.3 Función sin return

```
# Una función sin return retorna None implícitamente
def imprimir_reporte(titulo, valor):
    """Imprime un reporte formateado"""
    print("=" * 40)
    print(f"{titulo}: S/ {valor:,.2f}")
    print("=" * 40)

resultado = imprimir_reporte("Ventas Totales", 150000)
print(f"Resultado: {resultado}")  # None
```

```
========================================
Ventas Totales: S/ 150,000.00
========================================
Resultado: None
```

3 Tipos de argumentos

3.1 Argumentos posicionales

```
def calcular_tasa_crecimiento(valor_inicial, valor_final, periodos):
    """Calcula la tasa de crecimiento anualizada"""
    tasa = ((valor_final / valor_inicial) ** (1 / periodos) - 1) * 100
    return tasa

# Los argumentos se asignan por posición
tasa = calcular_tasa_crecimiento(100000, 150000, 5)
print(f"Tasa de crecimiento: {tasa:.2f}%")
```

```
Tasa de crecimiento: 8.45%
```

3.2 Argumentos con nombre (keyword arguments)

```
# Se pueden pasar argumentos usando el nombre del parámetro
tasa = calcular_tasa_crecimiento(
    valor_inicial=100000,
    valor_final=150000,
    periodos=5
)
print(f"Tasa de crecimiento: {tasa:.2f}%")

# Se puede cambiar el orden
tasa = calcular_tasa_crecimiento(
    periodos=5,
    valor_final=150000,
    valor_inicial=100000
)
print(f"Tasa de crecimiento: {tasa:.2f}%")
```

```
Tasa de crecimiento: 8.45%
Tasa de crecimiento: 8.45%
```

3.3 Argumentos con valores por defecto

```
def calcular_interes_compuesto(capital, tasa=0.05, periodos=1):
    """
    Calcula interés compuesto

    Parámetros con valores por defecto:
    - tasa: 5% anual por defecto
    - periodos: 1 año por defecto
    """
    monto_final = capital * (1 + tasa) ** periodos
```

```
    return monto_final

# Usar todos los valores por defecto
monto1 = calcular_interes_compuesto(10000)
print(f"Monto 1: S/ {monto1:,.2f}")

# Especificar solo algunos argumentos
monto2 = calcular_interes_compuesto(10000, tasa=0.08)
print(f"Monto 2: S/ {monto2:,.2f}")

# Especificar todos los argumentos
monto3 = calcular_interes_compuesto(10000, 0.10, 3)
print(f"Monto 3: S/ {monto3:,.2f}")
```

```
Monto 1: S/ 10,500.00
Monto 2: S/ 10,800.00
Monto 3: S/ 13,310.00
```

3.4 Argumentos variables (*args)

```
def calcular_promedio(*valores):
    """Calcula el promedio de un número variable de argumentos"""
    if len(valores) == 0:
        return 0

    total = sum(valores)
    promedio = total / len(valores)
    return promedio

# Diferente número de argumentos
prom1 = calcular_promedio(10, 20, 30)
print(f"Promedio 1: {prom1:.2f}")

prom2 = calcular_promedio(15, 25, 35, 45, 55)
print(f"Promedio 2: {prom2:.2f}")

prom3 = calcular_promedio(100)
print(f"Promedio 3: {prom3:.2f}")
```

```
Promedio 1: 20.00
Promedio 2: 35.00
Promedio 3: 100.00
```

3.5 Argumentos variables con nombre (**kwargs)

```python
def crear_reporte_economico(**indicadores):
    """Crea un reporte con indicadores económicos variables"""
    print("REPORTE ECONÓMICO")
    print("=" * 40)

    for nombre, valor in indicadores.items():
        print(f"{nombre.replace('_', ' ').title()}: {valor}")

    print("=" * 40)

# Diferentes indicadores cada vez
crear_reporte_economico(
    pib=242632,
    inflacion=3.5,
    desempleo=7.2
)

print()

crear_reporte_economico(
    pib=242632,
    inflacion=3.5,
    desempleo=7.2,
    tipo_cambio=3.75,
    reservas=74500
)
```

```
REPORTE ECONÓMICO
========================================
Pib: 242632
Inflacion: 3.5
Desempleo: 7.2
========================================

REPORTE ECONÓMICO
========================================
Pib: 242632
Inflacion: 3.5
Desempleo: 7.2
Tipo Cambio: 3.75
Reservas: 74500
========================================
```

4 Documentación de funciones

4.1 Docstrings

```python
def calcular_vpn(flujos, tasa_descuento):
    """
    Calcula el Valor Presente Neto (VPN) de un proyecto.

    Parámetros:
    -----------
    flujos : list
        Lista de flujos de caja. El primer elemento debe ser la inversión inicial (ne
    tasa_descuento : float
        Tasa de descuento anual (ej: 0.12 para 12%).

    Retorna:
    --------
    float
        El VPN del proyecto.

    Ejemplo:
    --------
    >>> flujos = [-100000, 30000, 40000, 50000]
    >>> vpn = calcular_vpn(flujos, 0.10)
    >>> print(f"VPN: {vpn:.2f}")
    """
    vpn = 0
    for t, flujo in enumerate(flujos):
        vpn += flujo / ((1 + tasa_descuento) ** t)

    return vpn

# Ver la documentación
print(calcular_vpn.__doc__)

# Usar la función
proyecto = [-100000, 30000, 40000, 50000]
vpn = calcular_vpn(proyecto, 0.12)
print(f"\nVPN del proyecto: S/ {vpn:,.2f}")
```

```
Calcula el Valor Presente Neto (VPN) de un proyecto.

Parámetros:
-----------
flujos : list
    Lista de flujos de caja. El primer elemento debe ser la inversión inicial (negati
tasa_descuento : float
    Tasa de descuento anual (ej: 0.12 para 12%).

Retorna:
--------
float
```

```
    El VPN del proyecto.

Ejemplo:
--------
>>> flujos = [-100000, 30000, 40000, 50000]
>>> vpn = calcular_vpn(flujos, 0.10)
>>> print(f"VPN: {vpn:.2f}")


VPN del proyecto: S/ -5,737.52
```

4.2 Anotaciones de tipo (Type hints)

```python
def calcular_elasticidad(precio_inicial: float,
                         precio_final: float,
                         cantidad_inicial: float,
                         cantidad_final: float) -> float:
    """
    Calcula la elasticidad precio de la demanda.

    Las anotaciones de tipo indican qué tipo de datos se espera.
    """
    var_porcentual_precio = ((precio_final - precio_inicial) / precio_inicial) * 100
    var_porcentual_cantidad = ((cantidad_final - cantidad_inicial) / cantidad_inicial

    elasticidad = var_porcentual_cantidad / var_porcentual_precio

    return elasticidad

# Usar la función
elast = calcular_elasticidad(100.0, 110.0, 1000.0, 950.0)
print(f"Elasticidad: {elast:.2f}")
```

```
Elasticidad: -0.50
```

5 Scope de variables

5.1 Variables locales vs globales

```python
# Variable global
tasa_igv = 0.18

def calcular_precio_con_igv(precio_base):
    """Calcula precio con IGV"""
    # precio_base es variable local
    # tasa_igv es variable global (accesible)
    precio_final = precio_base * (1 + tasa_igv)
    return precio_final
```

```
precio = calcular_precio_con_igv(100)
print(f"Precio con IGV: S/ {precio:.2f}")

# Modificar variables globales (no recomendado)
contador_llamadas = 0

def funcion_con_contador():
    """Incrementa contador global"""
    global contador_llamadas
    contador_llamadas += 1
    print(f"Función llamada {contador_llamadas} veces")

funcion_con_contador()
funcion_con_contador()
funcion_con_contador()
```

```
Precio con IGV: S/ 118.00
Función llamada 1 veces
Función llamada 2 veces
Función llamada 3 veces
```

5.2 Buenas prácticas

```
# Mejor práctica: retornar valores en lugar de modificar globales
def calcular_estadisticas(datos, contador=0):
    """Calcula estadísticas y actualiza contador"""
    contador += 1

    estadisticas = {
        'promedio': sum(datos) / len(datos),
        'maximo': max(datos),
        'minimo': min(datos),
        'llamadas': contador
    }

    return estadisticas, contador

ventas = [45000, 52000, 48000, 55000]
contador = 0

stats1, contador = calcular_estadisticas(ventas, contador)
print(f"Estadísticas 1: {stats1}")

stats2, contador = calcular_estadisticas([60000, 65000], contador)
print(f"Estadísticas 2: {stats2}")
```

```
Estadísticas 1: {'promedio': 50000.0, 'maximo': 55000, 'minimo': 45000, 'llamadas': 1
Estadísticas 2: {'promedio': 62500.0, 'maximo': 65000, 'minimo': 60000, 'llamadas': 2
```

6 Funciones lambda

Las funciones lambda son funciones anónimas pequeñas y de una sola línea.

6.1 Sintaxis básica

```python
# Función lambda simple
cuadrado = lambda x: x ** 2
print(f"Cuadrado de 5: {cuadrado(5)}")

# Equivalente a:
def cuadrado_normal(x):
    return x ** 2

# Lambda con múltiples argumentos
suma = lambda a, b: a + b
print(f"Suma: {suma(10, 20)}")

multiplicacion = lambda a, b, c: a * b * c
print(f"Multiplicación: {multiplicacion(2, 3, 4)}")
```

```
Cuadrado de 5: 25
Suma: 30
Multiplicación: 24
```

6.2 Aplicaciones económicas

```python
# Calcular IGV
calcular_igv = lambda precio: precio * 0.18
igv = calcular_igv(1000)
print(f"IGV: S/ {igv:.2f}")

# Calcular precio con IGV
precio_con_igv = lambda precio: precio * 1.18
precio_final = precio_con_igv(1000)
print(f"Precio con IGV: S/ {precio_final:.2f}")

# Calcular variación porcentual
var_porcentual = lambda inicial, final: ((final - inicial) / inicial) * 100
variacion = var_porcentual(100, 115)
print(f"Variación: {variacion:+.2f}%")

# Clasificar por tamaño
clasificar_empresa = lambda ventas: 'Grande' if ventas > 10000000 \
                                    else 'Mediana' if ventas > 1000000 \
                                    else 'Pequeña'

print(f"Empresa 1: {clasificar_empresa(500000)}")
```

```
print(f"Empresa 2: {clasificar_empresa(5000000)}")
print(f"Empresa 3: {clasificar_empresa(50000000)}")
```

```
IGV: S/ 180.00
Precio con IGV: S/ 1180.00
Variación: +15.00%
Empresa 1: Pequeña
Empresa 2: Mediana
Empresa 3: Grande
```

6.3 Lambdas en estructuras de datos

```
# Diccionario de funciones
operaciones = {
    'suma': lambda a, b: a + b,
    'resta': lambda a, b: a - b,
    'multiplicacion': lambda a, b: a * b,
    'division': lambda a, b: a / b if b != 0 else None
}

resultado = operaciones['suma'](10, 5)
print(f"Suma: {resultado}")

resultado = operaciones['division'](10, 2)
print(f"División: {resultado}")

# Lista de funciones de transformación
transformaciones = [
    lambda x: x * 1.18,  # Agregar IGV
    lambda x: x * 0.95,  # Descuento 5%
    lambda x: round(x, 2)  # Redondear
]

precio = 100
for transformacion in transformaciones:
    precio = transformacion(precio)
    print(f"Precio: S/ {precio:.2f}")
```

```
Suma: 15
División: 5.0
Precio: S/ 118.00
Precio: S/ 112.10
Precio: S/ 112.10
```

7 Funciones map y filter

7.1 Función map()

La función `map()` aplica una función a cada elemento de un iterable.

```python
# Sintaxis: map(funcion, iterable)

# Aplicar IGV a lista de precios
precios = [100, 200, 150, 300, 250]

# Con lambda
precios_con_igv = list(map(lambda x: x * 1.18, precios))
print(f"Precios con IGV: {precios_con_igv}")

# Con función definida
def agregar_igv(precio):
    return precio * 1.18

precios_con_igv = list(map(agregar_igv, precios))
print(f"Precios con IGV: {precios_con_igv}")

# Map con múltiples iterables
precios_2022 = [100, 105, 110, 108, 112]
precios_2023 = [105, 110, 115, 112, 118]

variaciones = list(map(
    lambda p1, p2: ((p2 - p1) / p1) * 100,
    precios_2022,
    precios_2023
))

print("\nVariaciones de precios:")
for i, var in enumerate(variaciones, 1):
    print(f"Producto {i}: {var:+.2f}%")
```

```
Precios con IGV: [118.0, 236.0, 177.0, 354.0, 295.0]
Precios con IGV: [118.0, 236.0, 177.0, 354.0, 295.0]

Variaciones de precios:
Producto 1: +5.00%
Producto 2: +4.76%
Producto 3: +4.55%
Producto 4: +3.70%
Producto 5: +5.36%
```

7.2 Función filter()

La función `filter()` filtra elementos que cumplen una condición.

```python
# Sintaxis: filter(funcion, iterable)

# Filtrar ventas mayores a 50000
ventas = [45000, 52000, 48000, 55000, 60000, 47000, 58000]
```

```python
ventas_altas = list(filter(lambda x: x > 50000, ventas))
print(f"Ventas altas: {ventas_altas}")

# Filtrar empresas por tamaño
empresas = [
    {'nombre': 'Empresa A', 'ventas': 500000},
    {'nombre': 'Empresa B', 'ventas': 5000000},
    {'nombre': 'Empresa C', 'ventas': 50000000},
    {'nombre': 'Empresa D', 'ventas': 800000},
]

grandes_empresas = list(filter(
    lambda e: e['ventas'] > 10000000,
    empresas
))

print("\nGrandes empresas:")
for empresa in grandes_empresas:
    print(f"  {empresa['nombre']}: S/ {empresa['ventas']:,}")

# Filtrar valores válidos (positivos)
datos = [100, -5, 200, 0, 150, -10, 300]
datos_validos = list(filter(lambda x: x > 0, datos))
print(f"\nDatos válidos: {datos_validos}")
```

```
Ventas altas: [52000, 55000, 60000, 58000]

Grandes empresas:
  Empresa C: S/ 50,000,000

Datos válidos: [100, 200, 150, 300]
```

7.3 Combinación de map y filter

```python
# Proceso de datos: filtrar y transformar
ventas_brutas = [45000, -1000, 52000, 48000, 0, 55000, -500, 60000]

# 1. Filtrar ventas válidas (positivas)
ventas_validas = filter(lambda x: x > 0, ventas_brutas)

# 2. Aplicar descuento del 5%
ventas_netas = map(lambda x: x * 0.95, ventas_validas)

# 3. Convertir a lista
resultado = list(ventas_netas)

print(f"Ventas brutas: {ventas_brutas}")
```

```
print(f"Ventas netas (válidas con descuento): {resultado}")

# Todo en una línea
resultado_compacto = list(map(
    lambda x: x * 0.95,
    filter(lambda x: x > 0, ventas_brutas)
))

print(f"Resultado compacto: {resultado_compacto}")
```

```
Ventas brutas: [45000, -1000, 52000, 48000, 0, 55000, -500, 60000]
Ventas netas (válidas con descuento): [42750.0, 49400.0, 45600.0, 52250.0, 57000.0]
Resultado compacto: [42750.0, 49400.0, 45600.0, 52250.0, 57000.0]
```

8 Módulo NumPy

NumPy (Numerical Python) es la biblioteca fundamental para computación científica en Python.

8.1 Instalación e importación

```
# Instalación: pip install numpy
import numpy as np
```

8.2 Creación de arrays

```
# Array desde lista
lista = [1, 2, 3, 4, 5]
array = np.array(lista)
print(f"Array: {array}")
print(f"Tipo: {type(array)}")

# Array multidimensional
matriz = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(f"\nMatriz:\n{matriz}")

# Arrays especiales
zeros = np.zeros((3, 4))  # Matriz de ceros
print(f"\nCeros:\n{zeros}")

ones = np.ones((2, 3))  # Matriz de unos
print(f"\nUnos:\n{ones}")

identidad = np.eye(4)  # Matriz identidad
print(f"\nIdentidad:\n{identidad}")

# Array con valores aleatorios
```

```
aleatorios = np.random.rand(3, 3)  # Uniformes entre 0 y 1
print(f"\nAleatorios:\n{aleatorios}")

# Rangos
rango = np.arange(0, 10, 2)  # Similar a range()
print(f"\nRango: {rango}")

linspace = np.linspace(0, 1, 5)  # 5 valores igualmente espaciados
print(f"Linspace: {linspace}")
```

```
Array: [1 2 3 4 5]
Tipo: <class 'numpy.ndarray'>

Matriz:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Ceros:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

Unos:
[[1. 1. 1.]
 [1. 1. 1.]]

Identidad:
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

Aleatorios:
[[0.85527655 0.00701474 0.04020534]
 [0.23155817 0.25489162 0.18235791]
 [0.67604556 0.77577792 0.72982946]]

Rango: [0 2 4 6 8]
Linspace: [0.   0.25 0.5  0.75 1.  ]
```

8.3 Operaciones con arrays

```
# Operaciones elemento por elemento
a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])
```

```
print(f"Suma: {a + b}")
print(f"Resta: {a - b}")
print(f"Multiplicación: {a * b}")
print(f"División: {b / a}")
print(f"Potencia: {a ** 2}")

# Operaciones con escalares
print(f"\nArray * 2: {a * 2}")
print(f"Array + 10: {a + 10}")

# Funciones universales
precios = np.array([100, 150, 200, 175, 225])
print(f"\nPrecios: {precios}")
print(f"Raíz cuadrada: {np.sqrt(precios)}")
print(f"Logaritmo: {np.log(precios)}")
print(f"Exponencial: {np.exp([0.1, 0.2, 0.3])}")
```

```
Suma: [11 22 33 44 55]
Resta: [ -9 -18 -27 -36 -45]
Multiplicación: [ 10  40  90 160 250]
División: [10. 10. 10. 10. 10.]
Potencia: [ 1  4  9 16 25]

Array * 2: [ 2  4  6  8 10]
Array + 10: [11 12 13 14 15]

Precios: [100 150 200 175 225]
Raíz cuadrada: [10.         12.24744871 14.14213562 13.22875656 15.        ]
Logaritmo: [4.60517019 5.01063529 5.29831737 5.16478597 5.4161004 ]
Exponencial: [1.10517092 1.22140276 1.34985881]
```

8.4 Estadísticas con NumPy

```
# Datos de ventas mensuales
ventas = np.array([45000, 52000, 48000, 55000, 60000, 58000,
                   62000, 59000, 65000, 70000, 68000, 75000])

print("ANÁLISIS ESTADÍSTICO DE VENTAS")
print("=" * 40)
print(f"Media: S/ {ventas.mean():,.2f}")
print(f"Mediana: S/ {np.median(ventas):,.2f}")
print(f"Desviación estándar: S/ {ventas.std():,.2f}")
print(f"Varianza: {ventas.var():,.2f}")
print(f"Mínimo: S/ {ventas.min():,.2f}")
print(f"Máximo: S/ {ventas.max():,.2f}")
print(f"Suma total: S/ {ventas.sum():,.2f}")
```

```python
# Percentiles
print(f"\nPercentil 25: S/ {np.percentile(ventas, 25):,.2f}")
print(f"Percentil 50: S/ {np.percentile(ventas, 50):,.2f}")
print(f"Percentil 75: S/ {np.percentile(ventas, 75):,.2f}")
```

```
ANÁLISIS ESTADÍSTICO DE VENTAS
========================================
Media: S/ 59,750.00
Mediana: S/ 59,500.00
Desviación estándar: S/ 8,564.70
Varianza: 73,354,166.67
Mínimo: S/ 45,000.00
Máximo: S/ 75,000.00
Suma total: S/ 717,000.00

Percentil 25: S/ 54,250.00
Percentil 50: S/ 59,500.00
Percentil 75: S/ 65,750.00
```

8.5 Operaciones con matrices

```python
# Datos de ventas por producto y región
# Filas: Productos, Columnas: Regiones
ventas_matriz = np.array([
    [10000, 15000, 12000],  # Producto A
    [8000, 12000, 9000],    # Producto B
    [15000, 18000, 14000]   # Producto C
])

print("Ventas por producto y región:")
print(ventas_matriz)

# Suma por filas (total por producto)
total_por_producto = ventas_matriz.sum(axis=1)
print(f"\nTotal por producto: {total_por_producto}")

# Suma por columnas (total por región)
total_por_region = ventas_matriz.sum(axis=0)
print(f"Total por región: {total_por_region}")

# Promedio por filas
promedio_por_producto = ventas_matriz.mean(axis=1)
print(f"\nPromedio por producto: {promedio_por_producto}")

# Transponer matriz
print(f"\nMatriz transpuesta:")
print(ventas_matriz.T)
```

```
Ventas por producto y región:
[[10000 15000 12000]
 [ 8000 12000  9000]
 [15000 18000 14000]]

Total por producto: [37000 29000 47000]
Total por región: [33000 45000 35000]

Promedio por producto: [12333.33333333  9666.66666667 15666.66666667]

Matriz transpuesta:
[[10000  8000 15000]
 [15000 12000 18000]
 [12000  9000 14000]]
```

8.6 Generación de números aleatorios

```python
# Semilla para reproducibilidad
np.random.seed(42)

# Distribución uniforme
uniforme = np.random.rand(5)
print(f"Uniforme [0,1]: {uniforme}")

# Distribución uniforme en rango
uniforme_rango = np.random.uniform(10, 20, 5)
print(f"Uniforme [10,20]: {uniforme_rango}")

# Distribución normal
normal = np.random.normal(100, 15, 1000)  # media=100, std=15
print(f"\nDistribución normal (muestra de 1000):")
print(f"  Media: {normal.mean():.2f}")
print(f"  Std: {normal.std():.2f}")

# Enteros aleatorios
enteros = np.random.randint(1, 100, 10)
print(f"\nEnteros aleatorios [1,100]: {enteros}")

# Aplicación: Simular rendimientos de acciones
dias = 252  # Días de trading en un año
rendimiento_diario_medio = 0.0008  # 0.08%
volatilidad = 0.02

rendimientos = np.random.normal(
    rendimiento_diario_medio,
    volatilidad,
    dias
)
```

```
precio_inicial = 100
precios = precio_inicial * (1 + rendimientos).cumprod()

print(f"\nSimulación de precios de acción:")
print(f"  Precio inicial: S/ {precio_inicial:.2f}")
print(f"  Precio final: S/ {precios[-1]:.2f}")
print(f"  Retorno anual: {((precios[-1]/precio_inicial - 1) * 100):.2f}%")
```

```
Uniforme [0,1]: [0.37454012 0.95071431 0.73199394 0.59865848 0.15601864]
Uniforme [10,20]: [11.5599452  10.58083612 18.66176146 16.01115012 17.08072578]

Distribución normal (muestra de 1000):
  Media: 100.29
  Std: 14.67

Enteros aleatorios [1,100]: [62  7 75 84 80  4  6 58 22 26]

Simulación de precios de acción:
  Precio inicial: S/ 100.00
  Precio final: S/ 185.73
  Retorno anual: 85.73%
```

9 Módulo Pandas

Pandas es la biblioteca principal para análisis y manipulación de datos en Python.

9.1 Instalación e importación

```
# Instalación: pip install pandas
import pandas as pd
import numpy as np
```

9.2 Series de Pandas

```
# Crear Series desde lista
ventas = pd.Series([45000, 52000, 48000, 55000, 60000])
print("Series de ventas:")
print(ventas)

# Series con índice personalizado
meses = ['Enero', 'Febrero', 'Marzo', 'Abril', 'Mayo']
ventas = pd.Series([45000, 52000, 48000, 55000, 60000], index=meses)
print("\nVentas por mes:")
print(ventas)

# Acceder a elementos
```

```
print(f"\nVentas de Marzo: S/ {ventas['Marzo']:,.2f}")
print(f"Ventas de Enero a Marzo:\n{ventas['Enero':'Marzo']}")

# Operaciones con Series
print(f"\nEstadísticas:")
print(f"  Media: S/ {ventas.mean():,.2f}")
print(f"  Máximo: S/ {ventas.max():,.2f}")
print(f"  Suma: S/ {ventas.sum():,.2f}")

# Operaciones elemento por elemento
ventas_con_descuento = ventas * 0.95
print(f"\nVentas con descuento 5%:")
print(ventas_con_descuento)
```

```
Series de ventas:
0    45000
1    52000
2    48000
3    55000
4    60000
dtype: int64

Ventas por mes:
Enero      45000
Febrero    52000
Marzo      48000
Abril      55000
Mayo       60000
dtype: int64

Ventas de Marzo: S/ 48,000.00
Ventas de Enero a Marzo:
Enero      45000
Febrero    52000
Marzo      48000
dtype: int64

Estadísticas:
  Media: S/ 52,000.00
  Máximo: S/ 60,000.00
  Suma: S/ 260,000.00

Ventas con descuento 5%:
Enero      42750.0
Febrero    49400.0
Marzo      45600.0
Abril      52250.0
Mayo       57000.0
```

```
dtype: float64
```

9.3 DataFrames

```python
# Crear DataFrame desde diccionario
datos = {
    'Producto': ['Laptop', 'Mouse', 'Teclado', 'Monitor'],
    'Cantidad': [15, 100, 50, 25],
    'Precio': [2500, 25, 80, 800],
    'Categoria': ['Computadoras', 'Accesorios', 'Accesorios', 'Computadoras']
}

df = pd.DataFrame(datos)
print("DataFrame de productos:")
print(df)

# Información del DataFrame
print(f"\nForma: {df.shape}")
print(f"Columnas: {df.columns.tolist()}")
print(f"\nTipos de datos:")
print(df.dtypes)

# Primeras y últimas filas
print(f"\nPrimeras 2 filas:")
print(df.head(2))
```

```
DataFrame de productos:
  Producto  Cantidad  Precio     Categoria
0   Laptop        15    2500  Computadoras
1    Mouse       100      25    Accesorios
2  Teclado        50      80    Accesorios
3  Monitor        25     800  Computadoras

Forma: (4, 4)
Columnas: ['Producto', 'Cantidad', 'Precio', 'Categoria']

Tipos de datos:
Producto     object
Cantidad      int64
Precio        int64
Categoria    object
dtype: object

Primeras 2 filas:
  Producto  Cantidad  Precio     Categoria
0   Laptop        15    2500  Computadoras
1    Mouse       100      25    Accesorios
```

9.4 Operaciones con DataFrames

```
# Crear columna calculada
df['Total'] = df['Cantidad'] * df['Precio']
print("\nDataFrame con columna Total:")
print(df)

# Seleccionar columnas
print(f"\nSolo Producto y Total:")
print(df[['Producto', 'Total']])

# Filtrar filas
productos_caros = df[df['Precio'] > 100]
print(f"\nProductos caros (>S/ 100):")
print(productos_caros)

# Agrupar y agregar
por_categoria = df.groupby('Categoria').agg({
    'Cantidad': 'sum',
    'Total': 'sum'
})
print(f"\nPor categoría:")
print(por_categoria)

# Estadísticas descriptivas
print(f"\nEstadísticas:")
print(df.describe())
```

```
DataFrame con columna Total:
  Producto  Cantidad  Precio     Categoria  Total
0   Laptop        15    2500  Computadoras  37500
1    Mouse       100      25    Accesorios   2500
2  Teclado        50      80    Accesorios   4000
3  Monitor        25     800  Computadoras  20000

Solo Producto y Total:
  Producto  Total
0   Laptop  37500
1    Mouse   2500
2  Teclado   4000
3  Monitor  20000

Productos caros (>S/ 100):
  Producto  Cantidad  Precio     Categoria  Total
0   Laptop        15    2500  Computadoras  37500
3  Monitor        25     800  Computadoras  20000

Por categoría:
```

```
             Cantidad  Total
Categoria
Accesorios         150   6500
Computadoras        40  57500

Estadísticas:
         Cantidad       Precio         Total
count    4.000000     4.000000      4.000000
mean    47.500000   851.250000  16000.000000
std     37.969286  1154.486719  16375.795146
min     15.000000    25.000000   2500.000000
25%     22.500000    66.250000   3625.000000
50%     37.500000   440.000000  12000.000000
75%     62.500000  1225.000000  24375.000000
max    100.000000  2500.000000  37500.000000
```

9.5 Lectura y escritura de datos

```python
# Crear DataFrame de ejemplo
datos_economicos = pd.DataFrame({
    'Año': [2019, 2020, 2021, 2022, 2023],
    'PIB': [230000, 225000, 245000, 250000, 258000],
    'Inflacion': [2.1, 1.8, 3.5, 8.3, 6.5],
    'Desempleo': [6.7, 13.0, 11.9, 7.8, 7.2]
})

print("Datos macroeconómicos:")
print(datos_economicos)

# Guardar como CSV
datos_economicos.to_csv('datos_macro.csv', index=False)
print("\nDatos guardados en 'datos_macro.csv'")

# Leer desde CSV
df_leido = pd.read_csv('datos_macro.csv')
print("\nDatos leídos desde CSV:")
print(df_leido)

# Guardar como Excel (requiere openpyxl)
# datos_economicos.to_excel('datos_macro.xlsx', index=False)
```

```
Datos macroeconómicos:
    Año     PIB  Inflacion  Desempleo
0  2019  230000        2.1        6.7
1  2020  225000        1.8       13.0
2  2021  245000        3.5       11.9
3  2022  250000        8.3        7.8
4  2023  258000        6.5        7.2
```

```
Datos guardados en 'datos_macro.csv'

Datos leídos desde CSV:
    Año     PIB  Inflacion  Desempleo
0  2019  230000        2.1        6.7
1  2020  225000        1.8       13.0
2  2021  245000        3.5       11.9
3  2022  250000        8.3        7.8
4  2023  258000        6.5        7.2
```

9.6 Aplicaciones económicas con Pandas

```python
# Análisis de series de tiempo
fechas = pd.date_range('2023-01-01', periods=12, freq='M')
ventas_mensuales = pd.Series(
    [45000, 52000, 48000, 55000, 60000, 58000,
     62000, 59000, 65000, 70000, 68000, 75000],
    index=fechas
)

print("Ventas mensuales:")
print(ventas_mensuales)

# Calcular variación porcentual
variacion = ventas_mensuales.pct_change() * 100
print(f"\nVariación porcentual:")
print(variacion)

# Promedio móvil
promedio_movil_3 = ventas_mensuales.rolling(window=3).mean()
print(f"\nPromedio móvil (3 meses):")
print(promedio_movil_3)

# Resumen estadístico
print(f"\nResumen:")
print(f"  Media: S/ {ventas_mensuales.mean():,.2f}")
print(f"  Crecimiento total: {((ventas_mensuales.iloc[-1]/ventas_mensuales.iloc[0] -
```

```
Ventas mensuales:
2023-01-31    45000
2023-02-28    52000
2023-03-31    48000
2023-04-30    55000
2023-05-31    60000
2023-06-30    58000
2023-07-31    62000
2023-08-31    59000
```

```
2023-09-30    65000
2023-10-31    70000
2023-11-30    68000
2023-12-31    75000
Freq: ME, dtype: int64

Variación porcentual:
2023-01-31          NaN
2023-02-28    15.555556
2023-03-31    -7.692308
2023-04-30    14.583333
2023-05-31     9.090909
2023-06-30    -3.333333
2023-07-31     6.896552
2023-08-31    -4.838710
2023-09-30    10.169492
2023-10-31     7.692308
2023-11-30    -2.857143
2023-12-31    10.294118
Freq: ME, dtype: float64

Promedio móvil (3 meses):
2023-01-31             NaN
2023-02-28             NaN
2023-03-31    48333.333333
2023-04-30    51666.666667
2023-05-31    54333.333333
2023-06-30    57666.666667
2023-07-31    60000.000000
2023-08-31    59666.666667
2023-09-30    62000.000000
2023-10-31    64666.666667
2023-11-30    67666.666667
2023-12-31    71000.000000
Freq: ME, dtype: float64

Resumen:
  Media: S/ 59,750.00
  Crecimiento total: 66.67%
```

10 Aplicaciones económicas

10.1 Aplicación 1: Calculadora financiera completa

```python
class CalculadoraFinanciera:
    """
    Clase con métodos para cálculos financieros comunes
    """
```

```
    @staticmethod
    def valor_presente(flujo_futuro, tasa, periodos):
        """Calcula el valor presente"""
        return flujo_futuro / ((1 + tasa) ** periodos)

    @staticmethod
    def valor_futuro(capital, tasa, periodos):
        """Calcula el valor futuro"""
        return capital * ((1 + tasa) ** periodos)

    @staticmethod
    def calcular_cuota(prestamo, tasa_mensual, num_cuotas):
        """Calcula la cuota de un préstamo (sistema francés)"""
        if tasa_mensual == 0:
            return prestamo / num_cuotas

        cuota = prestamo * (tasa_mensual * (1 + tasa_mensual) ** num_cuotas) / \
                ((1 + tasa_mensual) ** num_cuotas - 1)
        return cuota

    @staticmethod
    def tabla_amortizacion(prestamo, tasa_mensual, num_cuotas):
        """Genera tabla de amortización"""
        cuota = CalculadoraFinanciera.calcular_cuota(prestamo, tasa_mensual, num_cuot

        tabla = []
        saldo = prestamo

        for periodo in range(1, num_cuotas + 1):
            interes = saldo * tasa_mensual
            amortizacion = cuota - interes
            saldo -= amortizacion

            tabla.append({
                'Periodo': periodo,
                'Saldo_Inicial': saldo + amortizacion,
                'Cuota': cuota,
                'Interes': interes,
                'Amortizacion': amortizacion,
                'Saldo_Final': max(0, saldo)
            })

        return pd.DataFrame(tabla)

    @staticmethod
    def tir(flujos):
        """Calcula la TIR usando NumPy"""
```

```
        return np.irr(flujos) * 100 if hasattr(np, 'irr') else None

# Ejemplo de uso
calc = CalculadoraFinanciera()

# Valor presente
vp = calc.valor_presente(10000, 0.10, 3)
print(f"Valor presente: S/ {vp:,.2f}")

# Valor futuro
vf = calc.valor_futuro(10000, 0.08, 5)
print(f"Valor futuro: S/ {vf:,.2f}")

# Tabla de amortización
print("\nTabla de amortización:")
tabla = calc.tabla_amortizacion(50000, 0.015, 12)
print(tabla.to_string(index=False))

print(f"\nTotal pagado: S/ {(tabla['Cuota'].sum()):,.2f}")
print(f"Total intereses: S/ {tabla['Interes'].sum():,.2f}")
```

```
Valor presente: S/ 7,513.15
Valor futuro: S/ 14,693.28

Tabla de amortización:
 Periodo  Saldo_Inicial       Cuota    Interes  Amortizacion  Saldo_Final
       1   50000.000000 4583.999645 750.000000   3833.999645 46166.000355
       2   46166.000355 4583.999645 692.490005   3891.509640 42274.490715
       3   42274.490715 4583.999645 634.117361   3949.882285 38324.608430
       4   38324.608430 4583.999645 574.869126   4009.130519 34315.477911
       5   34315.477911 4583.999645 514.732169   4069.267477 30246.210435
       6   30246.210435 4583.999645 453.693157   4130.306489 26115.903946
       7   26115.903946 4583.999645 391.738559   4192.261086 21923.642860
       8   21923.642860 4583.999645 328.854643   4255.145002 17668.497857
       9   17668.497857 4583.999645 265.027468   4318.972177 13349.525680
      10   13349.525680 4583.999645 200.242885   4383.756760  8965.768920
      11    8965.768920 4583.999645 134.486534   4449.513112  4516.255808
      12    4516.255808 4583.999645  67.743837   4516.255808     0.000000

Total pagado: S/ 55,008.00
Total intereses: S/ 5,008.00
```

10.2 Aplicación 2: Análisis de portafolio con NumPy y Pandas

```
def analizar_portafolio(activos, pesos, rendimientos_esperados, matriz_covarianza):
    """
    Analiza un portafolio de inversión
```

```
    Parámetros:
    - activos: lista de nombres de activos
    - pesos: array de pesos (deben sumar 1)
    - rendimientos_esperados: array de rendimientos esperados
    - matriz_covarianza: matriz de covarianza de rendimientos
    """

    # Convertir a arrays de NumPy
    pesos = np.array(pesos)
    rendimientos = np.array(rendimientos_esperados)
    cov_matriz = np.array(matriz_covarianza)

    # Rendimiento esperado del portafolio
    rendimiento_portafolio = np.dot(pesos, rendimientos)

    # Varianza del portafolio
    varianza_portafolio = np.dot(pesos, np.dot(cov_matriz, pesos))

    # Desviación estándar (riesgo)
    riesgo_portafolio = np.sqrt(varianza_portafolio)

    # Ratio de Sharpe (asumiendo tasa libre de riesgo = 3%)
    tasa_libre_riesgo = 0.03
    sharpe = (rendimiento_portafolio - tasa_libre_riesgo) / riesgo_portafolio

    # Crear DataFrame con resultados
    resultados = pd.DataFrame({
        'Activo': activos,
        'Peso': pesos,
        'Rendimiento_Esperado': rendimientos
    })

    print("ANÁLISIS DE PORTAFOLIO")
    print("=" * 60)
    print("\nComposición del portafolio:")
    print(resultados.to_string(index=False))

    print(f"\nMétricas del portafolio:")
    print(f"  Rendimiento esperado: {rendimiento_portafolio*100:.2f}%")
    print(f"  Riesgo (desv. estándar): {riesgo_portafolio*100:.2f}%")
    print(f"  Ratio de Sharpe: {sharpe:.2f}")

    return {
        'rendimiento': rendimiento_portafolio,
        'riesgo': riesgo_portafolio,
        'sharpe': sharpe
    }
```

```
# Ejemplo de uso
activos = ['Acciones PE', 'Bonos', 'Inmuebles', 'Efectivo']
pesos = [0.40, 0.30, 0.20, 0.10]
rendimientos_esperados = [0.12, 0.06, 0.08, 0.02]

# Matriz de covarianza (ejemplo simplificado)
cov_matriz = [
    [0.04, 0.01, 0.02, 0.00],
    [0.01, 0.01, 0.01, 0.00],
    [0.02, 0.01, 0.02, 0.00],
    [0.00, 0.00, 0.00, 0.00]
]

resultados = analizar_portafolio(activos, pesos, rendimientos_esperados, cov_matriz)
```

```
ANÁLISIS DE PORTAFOLIO
============================================================

Composición del portafolio:
     Activo  Peso  Rendimiento_Esperado
Acciones PE   0.4                  0.12
      Bonos   0.3                  0.06
  Inmuebles   0.2                  0.08
   Efectivo   0.1                  0.02

Métricas del portafolio:
  Rendimiento esperado: 8.40%
  Riesgo (desv. estándar): 12.21%
  Ratio de Sharpe: 0.44
```

11 Ejercicios prácticos

11.1 Ejercicio 1: Sistema de predicción económica

```
def predecir_serie_tiempo(datos_historicos, metodo='promedio_movil', ventana=3):
    """
    Predice el siguiente valor de una serie de tiempo

    Métodos disponibles:
    - 'promedio_movil': Usa el promedio de los últimos n valores
    - 'tendencia_lineal': Ajusta una línea de tendencia
    - 'exponencial': Suavizado exponencial
    """

    datos = np.array(datos_historicos)

    if metodo == 'promedio_movil':
```

```
        if len(datos) < ventana:
            return datos.mean()
        prediccion = datos[-ventana:].mean()

    elif metodo == 'tendencia_lineal':
        # Ajustar línea de tendencia
        x = np.arange(len(datos))
        coef = np.polyfit(x, datos, 1)
        prediccion = coef[0] * len(datos) + coef[1]

    elif metodo == 'exponencial':
        # Suavizado exponencial simple
        alpha = 0.3
        prediccion = datos[-1]
        for i in range(len(datos) - 2, -1, -1):
            prediccion = alpha * datos[i] + (1 - alpha) * prediccion

    else:
        prediccion = datos[-1]

    return prediccion

# Datos históricos de PIB trimestral
pib_trimestral = [50000, 51000, 51500, 52000, 53000, 54000, 54500, 55000]

print("PREDICCIÓN DE PIB TRIMESTRAL")
print("=" * 60)
print(f"Datos históricos: {pib_trimestral}")

# Probar diferentes métodos
metodos = ['promedio_movil', 'tendencia_lineal', 'exponencial']

for metodo in metodos:
    pred = predecir_serie_tiempo(pib_trimestral, metodo)
    print(f"\nPredicción ({metodo}): {pred:,.0f}")

    # Calcular error con dato real (simulado)
    valor_real = 55500
    error = abs(valor_real - pred)
    error_porcentual = (error / valor_real) * 100
    print(f"  Error: {error:,.0f} ({error_porcentual:.2f}%)")
```

```
PREDICCIÓN DE PIB TRIMESTRAL
============================================================
Datos históricos: [50000, 51000, 51500, 52000, 53000, 54000, 54500, 55000]

Predicción (promedio_movil): 54,500
  Error: 1,000 (1.80%)
```

```
Predicción (tendencia_lineal): 55,893
  Error: 393 (0.71%)

Predicción (exponencial): 51,625
  Error: 3,875 (6.98%)
```

11.2 Ejercicio 2: Optimizador de producción

```python
def optimizar_produccion(productos, recursos_disponibles):
    """
    Optimiza la producción dados recursos limitados

    Parámetros:
    - productos: DataFrame con info de productos
    - recursos_disponibles: dict con recursos disponibles
    """

    # Calcular beneficio por unidad de recurso
    productos['beneficio_por_recurso'] = productos['beneficio'] / productos['recursos

    # Ordenar por beneficio por recurso (descendente)
    productos = productos.sort_values('beneficio_por_recurso', ascending=False)

    # Asignar producción
    recursos_usados = 0
    plan_produccion = []

    for idx, producto in productos.iterrows():
        recursos_requeridos = producto['recursos_requeridos']
        recursos_restantes = recursos_disponibles - recursos_usados

        if recursos_restantes >= recursos_requeridos:
            unidades = int(recursos_restantes / recursos_requeridos)
            recursos_usados += unidades * recursos_requeridos

            plan_produccion.append({
                'Producto': producto['nombre'],
                'Unidades': unidades,
                'Recursos': unidades * recursos_requeridos,
                'Beneficio': unidades * producto['beneficio']
            })

    # Crear DataFrame con plan
    plan_df = pd.DataFrame(plan_produccion)

    print("OPTIMIZACIÓN DE PRODUCCIÓN")
    print("=" * 70)
```

```
    print(f"\nRecursos disponibles: {recursos_disponibles:,} unidades")
    print(f"\nPlan de producción óptimo:")
    print(plan_df.to_string(index=False))

    print(f"\nResumen:")
    print(f"  Recursos utilizados: {plan_df['Recursos'].sum():,}")
    print(f"  Recursos disponibles: {recursos_disponibles - plan_df['Recursos'].sum()
    print(f"  Beneficio total: S/ {plan_df['Beneficio'].sum():,.2f}")

    return plan_df

# Ejemplo de uso
productos_df = pd.DataFrame({
    'nombre': ['Producto A', 'Producto B', 'Producto C', 'Producto D'],
    'beneficio': [100, 150, 80, 200],
    'recursos_requeridos': [20, 40, 15, 50]
})

plan = optimizar_produccion(productos_df, 1000)
```

```
OPTIMIZACIÓN DE PRODUCCIÓN
======================================================================

Recursos disponibles: 1,000 unidades

Plan de producción óptimo:
  Producto  Unidades  Recursos  Beneficio
Producto C        66       990       5280

Resumen:
  Recursos utilizados: 990
  Recursos disponibles: 10
  Beneficio total: S/ 5,280.00
```

12 Conlusión

En esta guía hemos explorado las funciones y módulos fundamentales de Python:

Funciones

Bloques de código reutilizables que realizan tareas específicas. Tipos de argumentos: posicionales, con nombre, con valores por defecto, variables.

Funciones lambda

Funciones anónimas de una línea, útiles para operaciones simples y rápidas.

map y filter

Funciones de orden superior que procesan iterables de manera eficiente.

NumPy

Biblioteca fundamental para computación numérica. Proporciona arrays multidimensionales y funciones matemáticas optimizadas.

Pandas

Biblioteca principal para análisis de datos. Proporciona estructuras de datos (Series y DataFrame) y herramientas para manipulación de datos.

13 Próximos pasos

Hemos completado las seis guías fundamentales de Python para economistas:

1. Introducción a Python
2. Variables, expresiones y statements
3. Objetos de Python (listas, diccionarios, tuplas)
4. Ejecución condicional
5. Iteraciones
6. Funciones y módulos

Estos conocimientos te proporcionan una base sólida para:

- Automatizar análisis económicos
- Procesar grandes volúmenes de datos
- Crear modelos y simulaciones
- Realizar análisis estadístico y econométrico
- Desarrollar herramientas de análisis personalizadas

14 Próximos pasos recomendados

Para continuar tu aprendizaje en Python aplicado a economía:

1. Profundizar en Pandas para análisis de datos
2. Aprender Matplotlib y Seaborn para visualización
3. Estudiar Statsmodels para econometría
4. Explorar Scikit-learn para machine learning
5. Practicar con datasets reales del BCRP, INEI, Banco Mundial

15 Recursos adicionales

- Python for Data Analysis (Wes McKinney)
- NumPy User Guide: numpy.org/doc
- Pandas Documentation: pandas.pydata.org
- Real Python: realpython.com
- Kaggle Learn: kaggle.com/learn

16 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python

8. 06 Funciones Con Python
9. 07 Dataframes Con Python
10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!